

В нашем проекте мы решаем задачу классификации на 100 классов. Для этого мы собрали свой датасет, при помощи парсинга текстов книг на английском языке. По фрагменту введённого текста наша модель предсказывает из ста авторов мировой классической литературы вероятность его авторства.

За время работы над проектом нашей командой было проведено множество экспериментов, как с генерацией признаков, так и с выбором лучшей модели с настройкой гиперпараметров.

Мы получили различные статистики наших текстовых данных, различные частотные словари, в том числе: количество знаков препинания, словарный запас, словари именованных сущностей, среднее количество слов в предложении и многое другое. Также мы получили векторы из текстов при помощи BoW, Tfidf, SentenceTransformer, TinyBert и Word2Vec.

С полученными признаками мы провели эксперименты с множеством классических моделей машинного обучения, таких как: Логистическая регрессия, метод k-ближайших соседей, Решающее дерево, Случайный лес, Метод опорных векторов, Наивный байесовский классификатор и различные бустинги, такие как CatBoost, LightGBM и XGBoost.

Наилучший результат удалось получить с использованием модели логистической регрессии (`solver='liblinear'`) с TF-IDF на уни и биграммах, с масштабированием `MaxAbsScaler`, с оборачиванием модели в `OneVsRestClassifier`. Точность модели (`accuracy`): 0.8043.

К прошлой контрольной точки проекта нами были внесены все правки по полученным замечаниям и выполнены некоторые улучшения.

А с момента последней контрольной точки, мы улучшили качество модели добавив ограничение максимального количества признаков до 10000 и `class_weight='balanced'` для учёта дисбаланса в классах, так как количество книг для каждого автора у нас разное. Таким образом точность модели возросла до 0.9394.

Подобрав гиперпараметры случайного леса, мы получили отдельно с его использованием метрику точности 0.8081. Затем обучив на этих моделях ансамбль нам удалось ещё улучшить наш результат до 0.9596.

```

from sklearn.ensemble import VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.preprocessing import MaxAbsScaler
from sklearn.pipeline import make_pipeline

# Определение базовых моделей
lr = LogisticRegression(solver='liblinear',
                        random_state=33,
                        class_weight='balanced')
rf = RandomForestClassifier(bootstrap = True,
                           random_state = 42,
                           class_weight = "balanced",
                           n_estimators = 1000)

vectorizer = TfidfVectorizer(ngram_range=(1, 2), max_features=10000)
ovr_lr = OneVsRestClassifier(lr)
ovr_rf = OneVsRestClassifier(rf)
scaler = MaxAbsScaler()
lr_pipe = make_pipeline(vectorizer, scaler, ovr_lr)
rf_pipe = make_pipeline(vectorizer, scaler, ovr_rf)

# Создание ансамбля моделей
ensemble_model = VotingClassifier(
    estimators=[('logistic', lr_pipe), ('random_forest', rf_pipe)],
    voting='soft'
)

# Обучение ансамбля
ensemble_model.fit(X_train, y_train)

```

```

{'accuracy': 0.9596,
 'precision_macro': 0.9722,
 'recall_macro': 0.92,
 'f1_macro': 0.9037,
 'precision_micro': 0.9596,
 'recall_micro': 0.9596,
 'f1_micro': 0.9596}

```

CPU times: user 18min 27s, sys: 6.93 s, total: 18min 34s
Wall time: 17min 46s

